

Jigsaw Puzzle Matching Using a Boundary-Centered Polar Encoding*

GERALD M. RADACK

AND

NORMAN I. BADLER

*Department of Computer and Information Science, University of Pennsylvania,
Philadelphia, Pennsylvania 19104*

Received January 2, 1981; revised August 6, 1981

A shape representation suitable for efficient matching (fitting together) of parts of boundaries of two-dimensional objects is described. This representation, which uses polar coordinate systems centered around curvature maxima and minima, is applied to the problem of fitting together curved jigsaw puzzle pieces. A program implementing these techniques is described.

1. INTRODUCTION

Much work has been done on the problem of matching two-dimensional closed curves—determining whether pairs of regions enclosed by the curves are similar in shape. In contrast, very little attention has been given to the problem of *boundary matching* (called “fitting” in [6]), i.e., determining whether *pieces* of closed or open curves are similar in shape, even though they may belong to globally different regions. This problem is approached by Freeman in [6], but only methods using the Freeman chain code [4] receive detailed discussion.

Most of the approaches to matching found in the literature involve either extraction of scalar features (such as moments, area, perimeter, and elongatedness) or computation of an internal representation such as the medial axis [3]. (Internal representations are those in which a portion of the encoding cannot be related to a single portion of the boundary, but depends on the entire boundary or several disjoint portions.) Internal representations are applicable only to closed curves. Scalar features (such as maximum curvature or distance between endpoints), while definable on open curves, give no clue as to which piece of a curve matches which piece of another curve, since they purposely transform the boundary data.

A natural testing ground for techniques of boundary matching is found in the *jigsaw puzzle problem*: Given a set of simply connected planar regions (puzzle pieces) whose only significant features are their boundaries, find how to rotate and translate each piece so that the pieces fit together into one region, with no holes or overlaps of pieces. The puzzle pieces are initially represented as sequences of Cartesian coordinates of closely spaced points, such as would be obtained from tracing the boundaries of the pieces on a data tablet. The presence of some noise in the initial representation is assumed. Note that we are not considering the use of any pictorial information on the puzzle pieces. Thus, the method presented here takes into account only one of the sources of information that humans use to fit puzzle pieces together. It

*This work was partially supported by NSF under Grant MCS78-07466.

should therefore not be viewed as an attempt to simulate the human puzzle assembly process.

We will restrict our domain to puzzles whose pieces are sufficiently unique that they can be put together without backtracking. Most real jigsaw puzzles have this property. Note that this uniqueness property depends not only on the geometry of the puzzle, but also on the amount of noise present in the initial representation. Although backtracking could certainly be incorporated in the approach presented here, it was omitted because it would add greatly to the complexity of the puzzle building programs without increasing our understanding of the problem. Of course, a puzzle building method might require backtracking to assemble a puzzle even if the above uniqueness property holds. In such a case, the method's matching criteria would have to be considered too inexact or tolerant of error. The goal here is to design matching criteria that are as accurate as possible, thus obviating backtracking except where it is demanded by the nature of the puzzle.

2. BOUNDARY MATCHING

Boundary matching between pairs of curves can be thought of as divided into three phases, each of which corresponds to one of the questions: (1) "Where to start?," (2) "Which way to go?," and (3) "How far to go?" These questions are explained below.

(1) "Where to start?" Find points on curves 1 and 2 that correspond. These will be called *match points*.

(2) "Which way to go?" Find the orientation of curve 2 with respect to curve 1 that allows the best match. One natural way to specify this is as a rotation around the match point. The angle of this rotation will be called the *match angle*.

(3) "How far to go?" Determine what are the matching segments of curves 1 and 2. The matching segments must be connected boundary segments containing the match points. The length of these segments (they should be roughly the same) will be called the *match length*.

The answer to one question will affect the answer to the others (see Fig. 1). Therefore, the order in which the questions are answered is important. Ideally, we would like to search the space of all possible triples of answers for the one that gives the best match; however, this is not practical. Therefore, we must decide on an order for answering them. In the work described here, the questions are answered in the order listed.

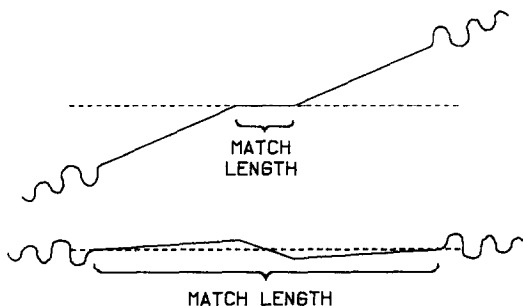


FIG. 1. Two different match angle/match length pairs are shown for the same curve pieces. The choice of one parameter will dictate the other. One curve piece is shown solid; the other is dotted.

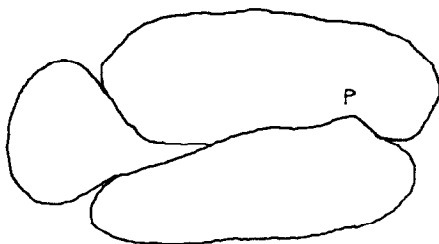


FIG. 2. This is a jigsaw puzzle that could not have been put together by a program relying on boundary segmentation. The presence of a point of local curvature maximum/minimum at P will allow the puzzle to be put together by the method presented here.

Freeman and Garder [7] chose to answer question (3) first, then (1) and (2). They split each curve at points where sharp corners occurred. The resulting segments were chain coded and then matched using a variety of scalar features such as length (along curve) of segment, distance between endpoints, and area between the line connecting the endpoints and the curve. For curves that were similar in all features, they identified an endpoint of one curve with an endpoint of the other (making these points the match points). The curves were then oriented so that the line between the endpoints of one curve was parallel to the line between the endpoints of the other. Because they found the match length first, they were forced to split up the piece boundaries arbitrarily in a way that did not necessarily correspond to how the puzzle fit together. Thus, their program would not have been able to put together the puzzle shown in Fig. 2 because of the lack of sharp corners at which to segment the boundaries.

2.1 Match Points

Since the curvature at a point on a curve depends only on the shape of the curve in a small neighborhood of that point, the curvature at corresponding match points should be the same. In order to limit the searching required, candidates for match points are restricted to points where curvature is at a maximum or minimum, which are called *critical points*. (Note that we are considering curvature as a signed function. On a region boundary, a curvature maximum will look like a curvature minimum except that one will be convex and one will be concave. Which is which depends on whether the boundary is parameterized in a clockwise or counter-clockwise manner. For jigsaw puzzle matching, all boundaries should be parameterized in the same direction so as to avoid having to explicitly find the interiors of the pieces. We have adopted the convention of always parameterizing boundaries in a clockwise manner, thus making maxima convex and minima concave.) Sharp corners (where curvature is undefined in the strict mathematical sense) are included as critical points. Because algorithms for approximating curvature on quantized curves are extremely sensitive to noise [2], smoothing must be done, making it hard to distinguish between corners and critical points where the curve is smooth. For this reason, no attempt is made to distinguish between these two types of critical points.

2.2 Match Angle

Computing the match angle can be thought of as lining up the curves so that the chosen match points correspond, then rotating one curve around its match point

until the area between the curves within a fixed distance D of the match points is minimized. The arbitrary distance D must be used because we have not settled on the match length yet.

The match points and match angle determine the *match orientation*, which is obtained by putting the two match points together then rotating the second curve by the match angle.

2.3. Match Length

The match length can be found by tracing each curve (in the match orientation) out from the match point in both directions until the first point is found in each direction which is greater than a certain tolerance threshold (THRESH) from the other curve. These points where the common portions of the boundaries end will be called *splitoff points*. It would be desirable to reorient the pieces after this step, minimizing the area between the boundaries up to the splitoff points. This has not been done in the current implementation.

2.4 Match Quality

The quality of a match is dependent on three factors:

(1) Length of the matching curve segments. This is ignored by us and by Freeman and Garder.

(2) Similarity of the curve segments. We measure this by finding an approximation to the area between the curves (in the match orientation) within distance D of the match points. The cutoff at distance D prevents us from penalizing a match for having high match length (and thus more area between the curves just from noise). It is also convenient because of the method used for encoding curves, which is described in Section 4.

(3) External factors not relating to the match itself but to information that is known about the jigsaw puzzle as a whole. One might want a check to reject a match that results in the creation of a concave corner sharper than any convex corners on existing pieces, since merging pieces cannot create sharper convex corners than exist already. This factor could also be used to incorporate a preference for matches that reduce the total number of corners (assuming that corners can be accurately detected), or a bias against matches that result in odd-shaped pieces. We have incorporated a weighting factor to give preference to matches with higher curvature at the match point. It was found experimentally that matches along relatively straight boundary segments often occur for puzzle pieces that do not really fit together. (The non-fit was detected later in the puzzle building process when the two pieces—in the match orientation—were checked to ensure that they did not overlap anywhere.) Thus, by considering such matches after those with higher curvature, puzzle solution can be sped up considerably. This experience provides a good justification for doing the matching around curvature extrema. External factors were ignored by Freeman and Garder.

2.5. Use of Orientation

A local curve orientation can be associated with a curve based on the direction in which it is traced. For example, we could adopt the convention that the normal of a curve $c(t)$ is directed to the left as one travels along the curve with t increasing. (If

the curve is the boundary of a region, then we would want the curve to be parameterized so that the normal is directed to the exterior of the region.)

These orientations can be taken into account during matching. We assume in this paper that all matching is done on *complementary* curves. That is, curves will match locally if they have the same shape but opposite normals.

3. BACKGROUND: ENCODING METHODS

Many methods have been used to encode boundaries of two-dimensional regions. Pavlidis [10] has identified two major orthogonal classifications for the encoding methods: *external vs internal* and *shape domain vs scalar transform*.

External methods depend on global properties of the region or its boundary, whereas internal methods examine only local properties of the boundary. Shape domain techniques transform a picture into another picture, whereas scalar transform techniques reduce a picture to a set of scalar features.

An example of an internal scalar transform technique is the method of moments [1, 8]. An example of an internal space domain technique is the medial axis transformation [3]. External scalar transform techniques typically involve Fourier transforms of the boundary function.

Internal methods are inappropriate for the jigsaw puzzle problem because each bit of the encoding contains global information. This would make it difficult to determine whether two encoded regions have matching boundary segments. Internal methods are more useful for comparing entire regions.

Scalar transform techniques are unsuited for the jigsaw puzzle problem because they do not facilitate checking that two pieces do not overlap or leave gaps when they are put together in a certain way. They also do not allow the merging of two pieces once they have been found to fit.

This leaves us with external space domain methods. Among these are polygonal approximations, the Freeman chain code and the pattern sequence.

A polygonal approximation of a region boundary is encoded as a sequence of (x, y) coordinates representing consecutive vertices of a polygon. The polygon should be entirely within some specified tolerance of the actual region boundary. Polygonal approximations can be made arbitrarily accurate by increasing the number of vertices as necessary, but they cannot be compared directly. Two polygonal approximations of the same region may be quite different due to arbitrariness in the choice of vertices. (See Fig. 3.)

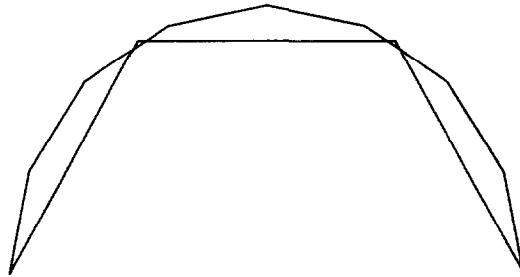


FIG. 3. Two different polygonal representations of the same curve (a circular arc) are shown. Note that there is no one-to-one correspondence between the vertices of the two representations.

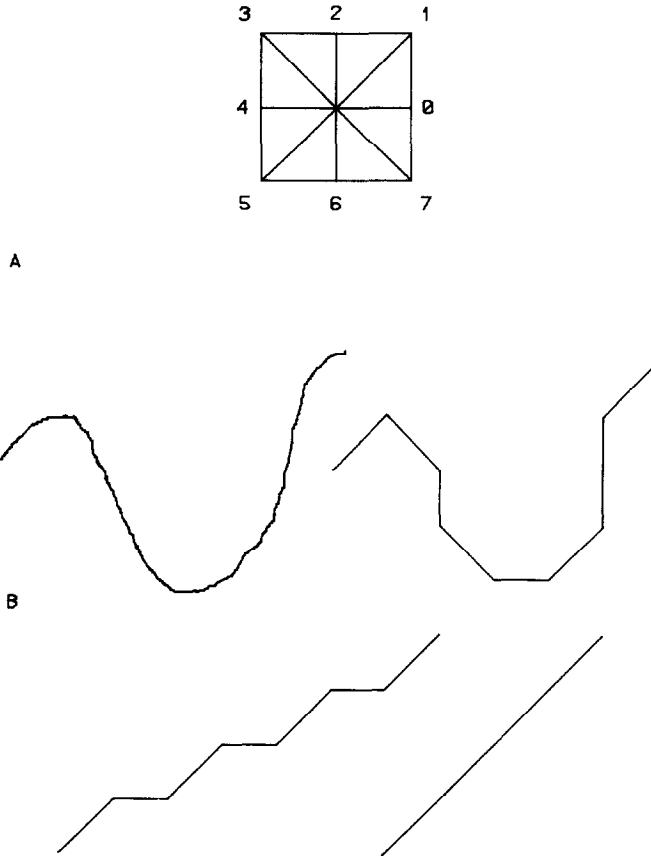


FIG. 4. *Freeman chain code*. The box shows the numbers representing the eight standard directions. (a) A curve and its encoding. (b) How the encoding of a straight line depends on the line's orientation.

Most external space domain techniques, including all those mentioned below, can be considered as special forms of polygonal approximations in which additional constraints are used to allow a more compact encoding and reduce the number of different possible encodings of a single piece.

A chain coding of a boundary [4] consists of a beginning point followed by a sequence of lines, each of which starts at the endpoint of the previous line. The lines are generally restricted to a few lengths and directions so that they can be represented by a small number indicating the direction and length. (See Fig. 4a.) A problem with this method is that a rotation will drastically affect the coding of a curve. (See Fig. 4b.) This means that matching cannot be done by directly comparing encodings. Instead, the encodings would have to be converted to some other form—probably Cartesian coordinates—before comparing. (Freeman has defined a “chain correlation” [5, 6] for comparing chain coded curves, but admits that before it can be used, the curves must have the same orientation and scale. This would have to be accomplished by using a process involving conversion to some Cartesian form.)

A pattern sequence representation of a boundary [9] consists of an encoding point inside the region and a sequence of distances from the point to the boundary along certain standard directions, usually multiples of a small angle. (See Fig. 5a.) If the

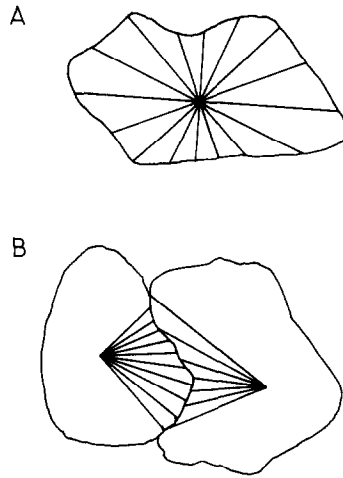


FIG. 5. *Pattern sequences.* (a) A pattern sequence for a region. (b) Pattern subsequences of the same boundary segment in two matching regions. Note that there is no direct relationship between the pattern sequences.

region is not angularly simple, several encoding points must be used. (A region R is angularly simple if there exists a point X such that R is the union of a set of convex regions, each containing X .)

The pattern sequence method is inappropriate for the jigsaw puzzle problem because two curves that fit together will have dissimilar pattern sequences. (See Fig. 5b.) This will again prevent matching by direct comparison of encodings.

This problem can be circumvented by taking distances from an encoding point along the boundary. If the same encoding point is used for two matching boundary segments, identical pattern sequences will be produced, thus allowing matching by direct comparison of encodings. (See Fig. 6.)

However, pattern sequences do not provide rotational invariance, since they find distances at standard angles. If we instead find angles at certain standard distances,

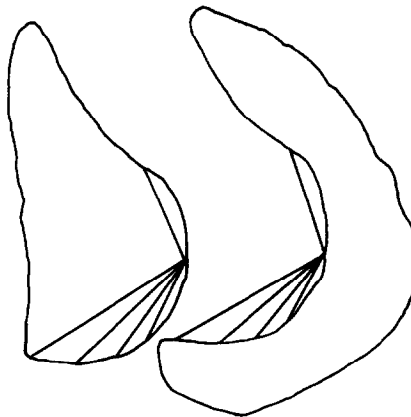


FIG. 6. If the same point along the common boundary is used as the center point for the pattern sequences of two matching regions, identical pattern subsequences will be obtained.

rotation invariance will be achieved, since rotation of the region will correspond to addition of a constant (mod 2π) to the angle. This is the method adopted here.

4. ENCODING METHOD USED

4.1. Representation of Curves

The portion of a curve near a critical point is represented as a sequence of points in a polar coordinate system whose origin is the critical point (called the *center point* for the coordinate system). This allows easy rotation around the critical point, which is essential for the match procedure described above. In order to provide a canonical representation for curve segments, a particular set of points are chosen: those whose distances from the critical point are multiples of a constant, DINCR. This allows us to represent a curve segment as a table of angles, which we call a "coding array." It also allows a match to be checked by just comparing angles, without interpolating. Note that for any distance, there are two angles, one for each of the two directions along the curve from the center point. The largest distance whose angles are stored in a given table is the distance D . It is assumed that adjacent critical points on a curve are within $2 \cdot D$ of each other, so that there is some overlap between the boundary segments encoded by adjacent coding arrays, thus assuring that no section of the boundary is left unencoded. If for a particular puzzle adjacent critical points are not within $2 \cdot D$ of each other, the problem can be corrected by simply increasing D .

Normally, there will be some overlap between the boundary segments encoded by adjacent coding arrays. When tracing along the boundary of an encoded piece, we need to know when to switch from one coding array to the next. A natural place to do this would be at the point between the two critical points at which curvature is 0. However, since zeroes of curvature are not needed anywhere else, we have decided for simplicity to make the switch so that the two coding arrays have the same number of significant entries (i.e., entries that would be used when tracing along the boundary). Since each entry of a coding array represents a specific distance from the center point, the switchover actually will occur at a point along the boundary equidistant from the two center points.

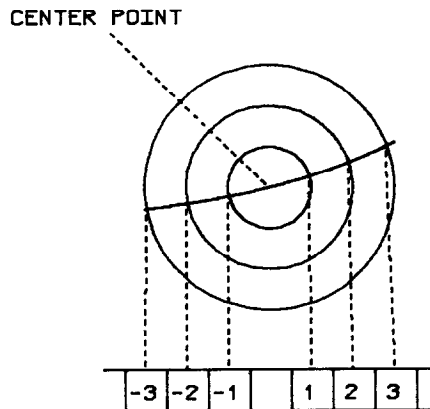


FIG. 7. The definition of a coding array.

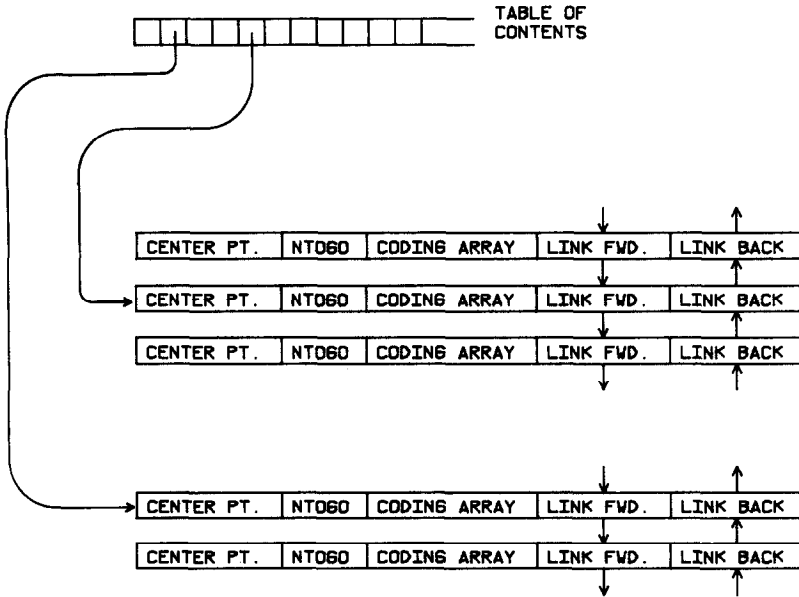


FIG. 8. A diagram of an encoded puzzle file. Each line represents one record in the file. A puzzle piece is stored as a doubly linked circular list of records, one for each critical point on the piece boundary. One record of each piece is pointed to by a table of contents entry. Note that NTOGO actually applies not to a single coding array (hence record) but to the transition between that coding array and the next coding array (in the forward direction).

Associated with each coding array are the Cartesian coordinates of its center point and an integer, NTOGO, which tells us when to switch from one coding array to the next when tracing along the curve.

Let $NDIST = D/DINCR$. Then a coding array can be represented as an array $C[-NDIST:NDIST]$, where $C[I]$ ($I \neq 0$) is a real number from the interval $[-\pi, \pi)$. (See Fig. 7.) If (CX, CY) are the coordinates of the center point, then point N of the coding array would have Cartesian coordinates

$$\text{cart}(C, N) = (CX, CY) + N \cdot DINCR \cdot (\cos(C[N]), \sin(C[N])). \quad (1)$$

It is clear from the above expression that $C[0]$ may be left undefined.

Figure 8 is a schematic diagram of the data structures used in encoding a jigsaw puzzle.

4.2. Encoding Algorithms

Locating the critical points. The critical points are located using an algorithm of Rosenfeld and Johnston [11]. An improvement of this algorithm is found in [12]; however, the original algorithm was found to work adequately.

Computing the polar coordinates. The polar coordinate entries of a coding array are computed as follows:

- (1) Start at the center point.
- (2) Get the next line segment of the boundary. If there is a point P on the line segment whose distance from the center point C is an integral multiple N of the

distance increment DINCR then

(a) Compute the angle Θ that the line from the center point to P makes with the x -axis.

(b) Store Θ in $C[N]$ if $C[N]$ is not already filled in.

(3) Go to step (2) unless $C[\text{NDIST}]$ has already been filled in.

The above process is then repeated going in the opposite direction along the boundary and filling in the negative array entries (e.g., $C[-N]$ if P is $N \cdot \text{DINCR}$ from the center point).

If P_1 and P_2 are the endpoints of a line segment with P_1 being closer to C along the boundary, then step 2 can be expressed as:

$$\begin{aligned} N_1 &\leftarrow \text{ceil}(d(P_1, C)/\text{DINCR}) \\ N_2 &\leftarrow \text{floor}(d(P_2, C)/\text{DINCR}) \\ \text{FOR } I = N_1 \text{ to } N_2 \text{ do} \\ \text{Compute } P &= \overline{P_1 P_2} \cap (\text{circle of radius } I \cdot \text{DINCR with} \\ &\quad \text{center at } C). \end{aligned}$$

The intersection computation is done directly without iteration.

Speed of this algorithm. If the distance between points in the input is less than DINCR, then we will have $N_1 \geq N_2$, meaning that at most one intersection computation will have to be done per execution of step 2. Therefore, consider step 2 to have unit cost. Then the total time required to encode a puzzle piece using the algorithm is

$$\sum_{p \in \text{CRITS}} (N_B(p) + N_F(p)).$$

Here N_B is the number of points along the boundary in the “backward” direction one must traverse in order to be at least distance D from point p . N_F is the similar value for the “forward” direction. CRITS is the set of critical points for the puzzle piece. If the average distance between points along the boundary is δ , it is reasonable to assume that all N_B and N_F values are $\leq 2D/\delta$. In this case, the cost of encoding a piece is less than

$$\#(\text{CRITS}) \cdot 2D/\delta,$$

where “ $\#$ ” represents the cardinality function.

Finding the switchover point for successive coding arrays (NTOGO). Because the switchover between two coding arrays occurs at the same distance from both center points, a single NTOGO can be used to specify it. Thus there will be the same number of NTOGO values as there are coding arrays; however, each NTOGO refers to a pair of consecutive coding arrays. The NTOGO between two consecutive coding arrays C and D can be computed by the method that follows. Note that in the following, $\text{cart}(C, I)$ means the point encoded by $C[I]$, given by Eq. (1).

(1) Start with N at 0.

(2) If $\text{cart}(C, N)$ is within $2 \cdot \text{DINCR}$ of $\text{cart}(D, -N)$ then stop; NTOGO is N .

(3) Increase N by 1 and go to step (2).

This algorithm takes time proportional to NDIST times the number of critical points.

5. DETAILS OF MATCHING METHOD

5.1. Overall Strategy

(1) For all pairs of critical points not on the same puzzle piece, estimate the quality of the best match having those points as match points, looking only at the curve segments within distance D of the critical points. This process, which we call *partial matching*, can be done cheaply by simply comparing the angles in the appropriate coding arrays.

(2) For the pair with the best quality estimate, see if the pieces really fit together, looking beyond distance D if necessary.

(3a) If so, merge the two pieces into one, making the appropriate changes to the data for the puzzle pieces. Compare any new critical points against existing critical points on other pieces for partial matches.

(3b) If a match does not exist, repeat step (2) using the pair with the next best quality estimate.

(4) If there is more than one piece remaining, go to step (2).

We will now describe the matching algorithms mentioned above. All additions and subtractions of angles are assumed to return principal values in the range $[-\pi, \pi)$.

5.2. Partial Matching

Partial matching is done by a weighted least-squares optimization. Let B and C be two coding arrays. Then

$$V(\Theta) = \sum_{\substack{-\text{NDIST} \leq I \leq \text{NDIST} \\ \text{NDIST} \neq 0}} |I| \cdot (B[I] - (C[-I] + \Theta))^2 \quad (2)$$

is a measure of the closeness of the curve segments encoded by B and C , when C is rotated by Θ . Using the absolute value of the differences instead of squaring would give a better approximation of the area between the curve segments, but would make it more difficult to minimize V . Note that $B[I]$ corresponds to $C[-I]$ because matching curves must be traced in opposite directions. The optimum match value for B and C (VMIN) is defined to be the minimum over Θ of $V(\Theta)$. The value of Θ achieving this minimum will be used as the match angle for this match. This minimum must occur when $dV/d\Theta = 0$. Some simple computation shows that this occurs when

$$\Theta = \frac{\sum |I| (B[I] - C[-I])}{\sum |I|}$$

or

$$\Theta = \frac{\sum |I| (B[I] - C[-I])}{\text{NDIST}(\text{NDIST} + 1)} \quad (3)$$

Weighting coefficient. The partial match values obtained above are weighted according to the sharpness of the critical points being compared. The weighting coefficient W is defined by

$$W = 1 + \text{WTFAC} \cdot \text{sharpness}.$$

WTFAC is a parameter used to control the variability of W and hence the extent to which priority is given to sharper critical points. The sharpness value ranges from 0 to 1 and is obtained by dividing the absolute value of the angle between $\text{cart}(C, -\text{WTINC})$, $\text{cart}(C, 0)$ and $\text{cart}(C, \text{WTINC})$ by 2π . WTINC is another parameter. A higher value of WTINC would cause a greater amount of “smoothing” in the sharpness computation.

The weighted partial match values ($W \cdot \text{VMIN}$) are used to determine the best partial match in step (2) of the match procedure. A lower value of $W \cdot \text{VMIN}$ is considered to indicate a better partial match.

5.3. Checking for Fit

Step (2) is done as follows: Put the boundaries in the match orientation. Find the common portions of the boundaries of the two pieces. (This will be discussed in detail below.) Determine if the remaining boundary portions intersect or come within THRESH of each other (except at their endpoints). If so, reject the match because either the pieces overlap (and thus do not fit together properly) or the pieces, when merged, would result in a piece with holes. This restriction against pieces with holes will not prevent the puzzle from being assembled; it will simply force the piece that fits in the hole to be merged with one of the surrounding pieces first.

The places where two curves in the match orientation split apart are defined to be the places closest to the match point (on either side) where one curve is greater than THRESH from the other curve. If the boundary segments encoded by the coding arrays overlap (as they should), consecutive vertex points will always be close together and we can assume that the splitoff occurs at a vertex. (If there are consecutive vertex points on a piece that are far apart, the splitoff could appear halfway between them if the matching piece veers away at this point.) This makes computation of the splitoff point faster because we can avoid calculating distances of line segments from line segments and can instead calculate distances of points from points. The vertex where splitoff occurs can easily be found by tracing along the matching boundaries with two pointers. Each pointer is, in turn, pushed forward as far as it can be without getting more than THRESH away from the other pointer. When neither pointer can be pushed forward further, the pointers are at the splitoff points.

5.4. Checking for Intersection

One possible way to check whether two curves intersect is to convert the curves back to Cartesian coordinates and use Algorithm 1 of Shamos and Hoey [13], which runs in $N \log N$ time (where N is the total number of line segments—hence vertices—making up the boundary segments being checked). Unfortunately, this algorithm cannot be adapted to also check whether the curves come within THRESH of each other. Alternatively, one can make use of the fact that two points on

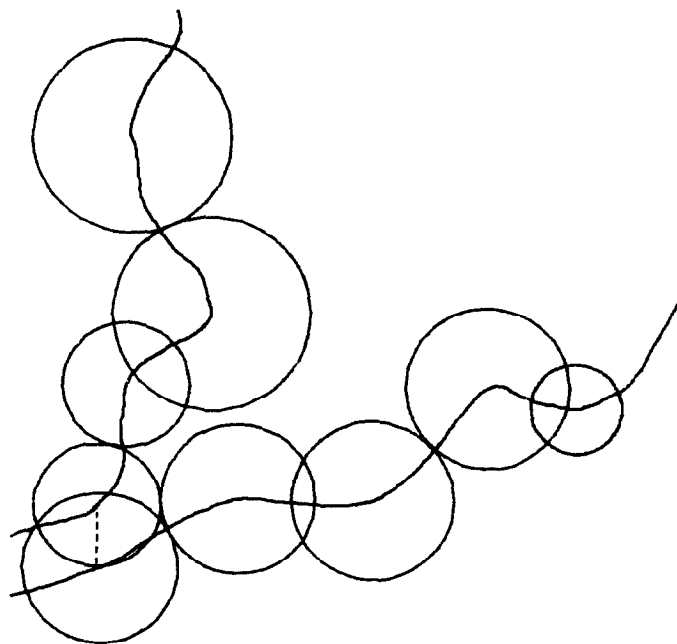


FIG. 9. Checking for curve intersections can be done on a coding array basis, only checking those pairs of arrays whose distance between center points is less than the sum of their extents. The circles represent the extents of the arrays (not the actual boundary segments encoded by the arrays, which will be somewhat larger); the dots represent center points. Only the pair of arrays whose center points are connected by a dotted line would have to be checked.

different curves can only be near each other if the center points of their respective coding arrays are near each other. Let the *extent* of a coding array be the maximum distance of any point coded by the array from the center point. This can easily be computed from the NTOGO for the array and the NTOGO for the previous array. Then by the triangle inequality we only have to check pairs of boundary segments whose coding arrays have center points no more distant than the sum of their extents. (See Fig. 9.) This method is of order N^2 in worst case performance; however, due to the nature of the pieces, this method may be faster in practice than the Shamos method.

6. CHOICE OF PARAMETERS

We now discuss how to choose the various parameters for an individual puzzle.

The first parameter that must be chosen is one not mentioned yet. It is called M and is used in the critical point finding algorithm [11] to control the amount of smoothing done there. The amount of smoothing will increase as M is increased. M should be set just low enough to enable all important critical points to be found.

The distance between encoded points is always at least DINCR (the increment in distance from the center point for points in a coding array) and normally does not exceed $2 \cdot \text{DINCR}$. It would thus be sensible to set DINCR to equal the average distance between the unencoded points in the original representation.

D (the distance of the outermost points in a coding array from the center point) must be large enough so that any adjacent critical points on a piece are within $2 \cdot D$

of each other, thereby ensuring that the entire piece is encoded by some coding array. Thus, D could be set to half the maximum distance between adjacent critical points.

By definition now, we have that $NDIST$, the number of points per direction in a coding array, is equal to $D/DINCR$.

The choices of $WTFAC$ and $WTINC$ are somewhat arbitrary. (Recall that $WTFAC$ determines the amount of priority given to sharpness of a critical point during partial matching. $WTINC$ affects the smoothing done in computing that sharpness.) We have found that $WTFAC$ must be at least 2.0 in order for the "correct" (sharp) critical points to be checked first. Increasing it beyond 3.0 does not seem to change the results much. A value of 5 seems to work nicely for $WTINC$.

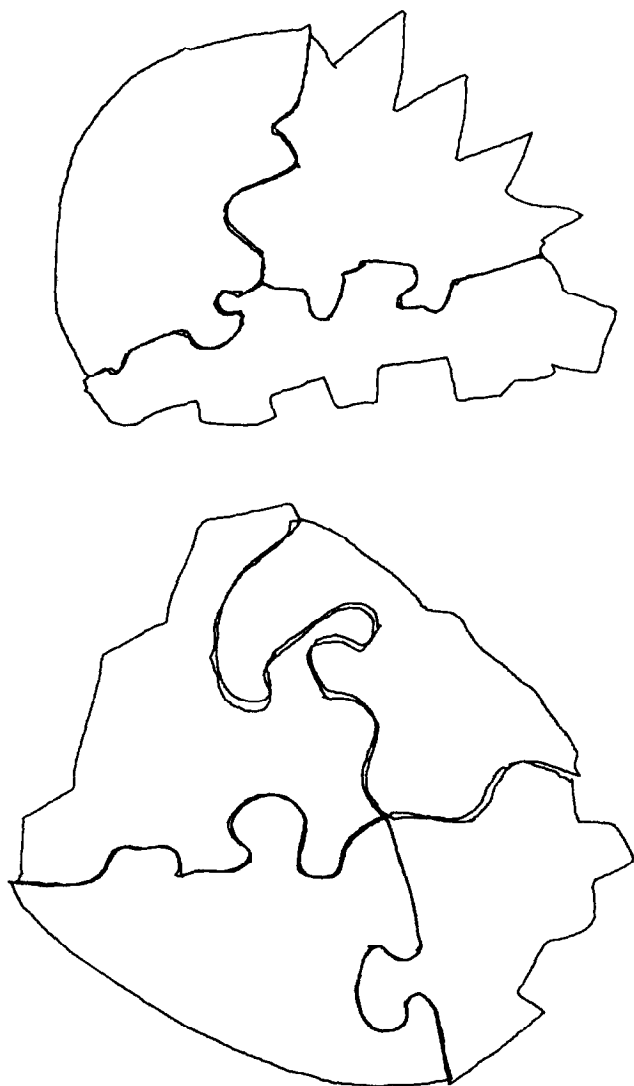


FIG. 10. Puzzles that were correctly put together.

THRESH (the distance apart that boundaries must get before splitoff is said to occur) must be at least equal to twice the maximum distance between encoded points in order for the splitoff finding algorithm to work.

7. RESULTS

The method presented was implemented and tested using several jigsaw puzzles. It was able to put together a number of small puzzles, including the puzzles shown in Fig. 10. Figure 11 shows a puzzle that was put together incorrectly with one set of parameters and correctly with higher THRESH values. (Two separate values of THRESH were used, one for finding splitoff and one for finding intersection.)

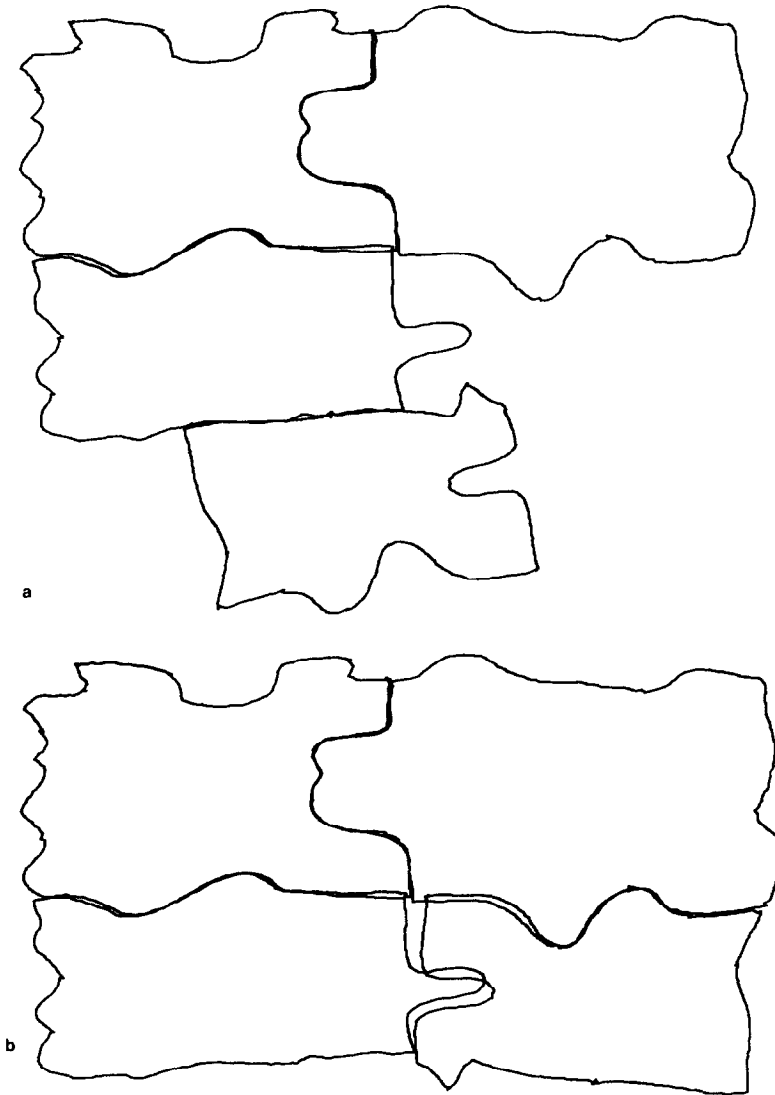


FIG. 11. The puzzles in (a) and (b) are identical. (a) Put together using a THRESH of 0.02 for finding splitoff and 0.01 for determining intersection. (b) Used a THRESH of 0.2 for splitoff and 0.4 for intersection. Although (a) was not put together as expected, it fits quite well as shown.

During the incorrect assembly, the correct partial matches were found and tried, but the last piece could not fit in the intended place because the previous piece was fit with a slightly wrong match angle. With larger THRESH values, the program accepted this match despite the errors.

This problem is essentially one of accumulation of small errors that eventually distort the puzzle being built up enough to prevent later pieces from fitting. The problem could be greatly reduced by correcting the match angle after the match length is computed. One possible way to do this would be to adjust the angle in order to minimize the area between the matching boundary portions. Another possibility would be to compute polar coordinates for all multiples of DINCR up to the match length and then use Eq. 3 to find an improved match angle.

7.1. Parameters Used

The original data tablet representations of the pieces had between 200 and 500 points per piece, spaced between 0.006 and 0.01 units apart. The coordinates of the points were within the range of -1.0 to 1.0 . M (a smoothing parameter used by the critical point finding algorithm [11]) was set to 15. DINCR was 0.01. NDIST was 25, meaning that D was 0.25. A THRESH value of 0.02 was used for finding the splitoff. A value of 0.01 was used for finding intersection of non-matching boundary portions. Higher values of these THRESH parameters (0.4 for splitoff and 0.2 for intersection) were also used with success, particularly on puzzles that were not as accurately drawn as others. WTFAC was 3.0 and WTINC was 5.

8. CONCLUSIONS

A new boundary encoding scheme has been described that facilitates boundary matching. This has been used to construct a method for solution of jigsaw puzzles that avoids unnecessary backtracking through the use of a very selective matching algorithm. This method has been implemented in a program that can put together actual jigsaw puzzles whose pieces have been traced on a data tablet.

Areas in which future research might proceed include:

1. Determining whether a better critical point finding algorithm can significantly improve or speed up puzzle fitting.
2. Investigating ways of refining the match angle after the match length has been chosen in order to make the matching curves fit as closely as possible.
3. Extending the algorithm to puzzles on non-flat surfaces (such as pottery pieces). The matching problem for such surfaces is still two-dimensional in nature, but three-dimensional coordinates would be needed for points along a boundary. A second coding array (per critical point) could be added for this purpose.
4. Finding a way to extend the idea of rotational invariance via polar encodings to surfaces of regions in three-space. The analogous jigsaw puzzle problem would be that of fitting three-dimensional regions to form one solid region.

REFERENCES

1. F. L. Alt, Digital pattern recognition by moments, *J. Assoc. Comput. Mach.* **11**, 1962, 240-258.
2. J. Bennett and J. Macdonald, On the measurement of curvature in a quantized environment, *IEEE Trans. Comput.* **C-24**, 1975, 803-820.
3. H. Blum, A transformation for extracting new descriptors of shape, in *Models for the Perception of Speech and Visual Form* (W. Wathen-Dunn, Ed.), pp. 362-380. MIT Press, Cambridge, Mass., 1967.

4. H. Freeman, On the encoding of arbitrary geometric configurations, *IEEE Trans. Electronic Comput.* **EC-10**, 1961, 260–268.
5. H. Freeman, Computer processing of line-drawing images, *Computing Surveys* **6**, 1974, 56–97.
6. H. Freeman, Shape description via the use of critical points, *Pattern Recognition* **10**, 1978, 159–166.
7. H. Freeman and L. Garder, Apictorial jigsaw puzzles: The computer solution of a problem in pattern recognition, *IEEE Trans. Electronic Comput.* **EC-13**, 1964, 118–127.
8. M. K. Hu, Visual pattern recognition by moment invariants, *IEEE Trans. Inform. Theory* **IT-8**, 1962, 179–187.
9. K. Maruyama, *A Study of Visual Shape Perception*, UIUCDCS-R-72-533, Department of Computer Science, University of Illinois, Urbana-Champaign, Ill., 1972.
10. T. Pavlidis, A review of algorithms for shape analysis, *Computer Graphics and Image Processing* **7**, 1978, 243–258.
11. A. Rosenfeld and E. Johnston, Angle detection on digital curves, *IEEE Trans. Comput.* **C-22**, 1973, 875–878.
12. A. Rosenfeld and J. Weszka, An improved method of angle detection on digital curves, *IEEE Trans. Comput.* **C-24**, 1975, 940–941.
13. M. Shamos and D. Hoey, Geometric Intersection Algorithms, 17th Annual Symposium on Foundations of Computer Science, Houston, 1976, pp. 208–215.